Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

NETPULSE: REAL-TIME NETWORK PACKET CAPTURE, PROTOCOL DECODING AND LIVE STREAMING DASHBOARD

24BKT0132 ADITYA PRATAP

B.Tech.

in

Computer Science and Engineering

(Specialization in Blockchain Technology)

School of Computer Science and Engineering

October 2026

ABSTRACT

Packet capture underpins network troubleshooting, performance monitoring and security analysis, yet student-level sniffers often break on modern dual-stack traffic. This project presents NetPulse, a lightweight capture, decode and visualisation pipeline for Linux built with C++ (libpcap), POSIX sockets, Node.js/Express and Socket.IO. It contains three C++ programs: a terminal analyser with BPF filtering, per-protocol statistics and hex dumps; an HTTP streamer that forwards decoded packets to a web backend; and a UDP DNS forwarder that reports every query it relays. A browser dashboard displays the stream live.

The work addresses three defects of a baseline libpcap sniffer. First, reading the IPv6 Next Header field directly mislabels Multicast Listener and Neighbor Discovery traffic as “Protocol 0”; NetPulse walks the chain of Hop-by-Hop, Routing, Fragment, Destination Options, Authentication and Mobility headers to expose the true upper-layer protocol, ICMPv6 type and ports. Second, Ethernet-only parsing fails on the Linux “any” pseudo-interface; the decoder instead selects the framing from the datalink type (Ethernet with 802.1Q, Linux cooked v1 and v2, loopback). Third, streaming packets to a backend on a captured interface causes a feedback storm; a kernel BPF expression and a user-space port check remove the loop, while 250 ms socket timeouts stop the capture thread from stalling.

The system is evaluated against the baseline in three load scenarios (50, 1,000 and 10,000 packets/s) using identification accuracy, backend amplification, kernel drop rate and traffic coverage, as mean and standard deviation of five runs, with ground truth cross-checked by tcpdump. The baseline identified none of the IPv6 extension-header packets and the proposed build all of them; the guard cut backend amplification from 36.8 to 1.0 at low load. Limitations include a throughput ceiling of about 1,800 packets/s in the test environment, non-first IPv6 fragments and the absence of flow reassembly; future work is outlined.

Certificate

This is to certify that the project report titled “NetPulse: Real-Time Network Packet Capture, Protocol Decoding and Live Streaming Dashboard” is a bonafide record of the work done by Aditya Pratap (24BKT0132) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Aditya Pratap
Name of the Student

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	i
1.	**INTRODUCTION**	1
	1.1 Background and motivation	1
	1.2 Problem statement	1
	1.3 Objectives	1
	1.4 Scope and limitations	2
	1.5 Mapping to course modules and the lab experiment	2
2.	**LITERATURE SURVEY**	2
	2.1 Overview of the existing protocol or scheme	2
	2.2 Review of related work	3
	2.3 Research gap and justification of novelty	4
3.	**PROPOSED SYSTEM**	4
	3.1 System architecture or topology	4
	3.2 Baseline mechanism description	5
	3.3 Proposed mechanism: algorithm, pseudocode and flowchart	6
	3.4 Novelty statement	8
	3.5 Assumptions and design decisions	8
4.	**IMPLEMENTATION**	9
	4.1 Tools, platform, versions and hardware	9
	4.2 Simulation or testbed setup	10
	4.3 Code structure and explanation of key modules	11
	4.4 Challenges faced and how they were solved	12
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	13
	5.1 Performance metrics and why they were chosen	13
	5.2 Scenarios and parameter values	14
	5.3 Results: tables and graphs, baseline vs proposed	15
	5.4 Analysis and discussion	16
	5.5 Critical evaluation: strengths, weaknesses and failure conditions	17
6.	**CONCLUSION AND FUTURE WORK**	18

Sl.No	Contents	Page No.
	6.1 Summary of work and key findings	18
	6.2 Limitations	18
	6.3 Future enhancements	19
7.	**REFERENCES**	19
	References (IEEE style)	19
	Appendix A: Important source code or configuration listings	20
	Appendix B: Additional screenshots, packet captures, raw result tables	22

1. INTRODUCTION

1.1 Background and motivation

Every layer of the Internet protocol suite leaves a visible trace on the wire, and the ability to capture and interpret those traces is the most direct way to understand how a network behaves. Packet capture is used to debug connectivity, to measure the load generated by applications, to verify that protocols are implemented correctly and to detect abnormal or hostile traffic. In a Computer Networks course the same ability turns abstract header diagrams into something that can be observed: a student can send a ping, a DNS query or a web request and immediately see the Ethernet, IP, transport and application fields that carry it.

Production analysers such as Wireshark are comprehensive but are desktop applications whose internals are hidden from the user, and simple command-line sniffers print text that is hard to share or watch live. A third difficulty is that many compact sniffers were written for IPv4 over Ethernet and quietly misbehave on today's dual-stack hosts. An idle Linux machine continuously emits IPv6 Multicast Listener Discovery reports and Neighbor Discovery messages, and these packets carry an IPv6 Hop-by-Hop extension header that a naive parser cannot see past. The motivation for this project is therefore to build a small, transparent and correct capture pipeline that exposes each layer, handles modern IPv6 traffic and presents the result in a live browser dashboard that can be demonstrated and shared.

1.2 Problem statement

A straightforward libpcap sniffer that assumes Ethernet framing and treats the IPv6 Next Header field as the transport protocol has three practical failure modes. (i) IPv6 packets that begin with extension headers are reported as "Protocol 0", so ICMPv6 control traffic, TCP and UDP carried behind those headers are misclassified and their ports are lost. (ii) Capturing on the Linux "any" pseudo-interface produces Linux cooked frames rather than Ethernet frames, so every offset is wrong. (iii) If the sniffer forwards each decoded packet to a web backend over the same host, the forwarding traffic is itself captured, which triggers further forwarding and produces a self-sustaining feedback storm.

The problem addressed in this project is to design, implement and evaluate a capture, decode and streaming system that (a) decodes IPv4, IPv6 and ARP traffic correctly across the link types that Linux can present, (b) reveals the real upper-layer protocol behind IPv6 extension header chains, (c) is immune to self-capture feedback, and (d) delivers a live view of the traffic without allowing a slow consumer to stall the capture loop.

1.3 Objectives

1. Implement a libpcap-based terminal analyser that decodes Ethernet (with 802.1Q), Linux cooked v1/v2 and loopback frames, IPv4, IPv6, ARP, TCP, UDP, ICMP and ICMPv6, with BPF filtering and capture statistics.
2. Implement an IPv6 extension header traversal routine that resolves the real upper-layer protocol for Hop-by-Hop, Routing, Fragment, Destination Options, Authentication and Mobility headers.

3. Implement a streaming path (C++ HTTP client, Node.js/Express backend, Socket.IO, browser dashboard) with bounded blocking time and automatic exclusion of the system's own traffic.
4. Implement a UDP DNS forwarder that parses the query name and type (RFC 1035) and reports each query to the dashboard backend.
5. Evaluate the proposed pipeline against a baseline sniffer using protocol-identification accuracy, backend amplification and kernel drop rate in three load scenarios, using repeated runs.

1.4 Scope and limitations

The scope is passive capture and decoding on a single Linux host. The decoders cover Ethernet II with 802.1Q, Linux cooked captures, loopback, IPv4, IPv6 with extension headers, ARP, TCP flags and ports, UDP ports, ICMP and ICMPv6 message types, and DNS query names. The project does not reassemble TCP streams, decrypt traffic, defragment IP packets, track flows or perform intrusion detection. The dashboard is intended for a local lab network and has no authentication or transport encryption. The programs use Linux-specific features (cooked capture, file capabilities); they were developed on Kali Linux and evaluated on Ubuntu 24.04 using the loopback interface.

1.5 Mapping to course modules and the lab experiment

The project exercises every layer taught in the course and the network-programming skills practised in the laboratory. Table 1 relates the work to the course topics.

Table 1: Mapping of the project to course topics

Course topic	Where it appears in the project	Lab skill exercised
Data link layer	Ethernet II framing, MAC addresses, 802.1Q VLAN tag, Linux cooked capture, loopback framing	Frame structure and EtherType demultiplexing
Network layer	IPv4 header and IHL, IPv6 header and extension header chain, ICMP and ICMPv6 types, ARP	Header parsing, address handling, protocol numbers
Transport layer	TCP flags and ports, UDP ports, well-known service mapping	Port-based multiplexing, TCP connection flags
Application layer	DNS query format and forwarding, HTTP POST and REST API, WebSocket push to the dashboard	Request/response protocols, JSON payloads
Network programming	libpcap capture, BPF filters, BSD sockets (TCP client, UDP server), timeouts, signal handling, threads	Socket API, concurrency, privilege handling

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

Packet capture model. On Linux a user-space program obtains raw frames through the libpcap library, which wraps the kernel's packet socket interface. McCanne and Jacobson's BSD Packet Filter introduced the idea of running a small filter program inside the kernel so that unwanted packets are discarded before they are copied to user space [1]. libpcap compiles human-readable filter expressions (for example "tcp port 80") into this form, delivers each accepted packet to a callback together with a header that records the captured length and the original length, and exposes counters for packets received and dropped by the kernel. The frame format delivered to the callback depends on the interface: the datalink type reported by libpcap distinguishes Ethernet, Linux cooked captures used for the "any" pseudo-interface, and loopback.

IPv6 header chain. IPv6 simplified the base header to a fixed 40 bytes and moved optional information into extension headers that form a chain, each header carrying a Next Header field that names the one that follows [2]. The extension headers handled by this project are Hop-by-Hop Options (0), Routing (43), Fragment (44), Destination Options (60), Authentication Header (51) and Mobility (135). Because the base header's Next Header field names the first extension header rather than the transport protocol, any device that wants to read TCP or UDP ports must traverse the chain. RFC 9098 lists the inability to find Layer 4 information as one of the operational difficulties caused by extension headers [3], and RFC 7112 requires that the first fragment of a packet contain the whole header chain precisely so that upper-layer information can be located [4]. Multicast Listener Discovery and Neighbor Discovery messages are carried in ICMPv6 [5], and MLD packets are sent with a Hop-by-Hop Router Alert option, which is why they are the first packets a naive parser misreports.

DNS. A DNS message begins with a 12-byte header followed by the question section, whose QNAME is a sequence of length-prefixed labels terminated by a zero byte, optionally using compression pointers, followed by a two-byte QTYPE [6]. Reading this structure is enough to report which names clients are resolving.

2.2 Review of related work

Table 2 summarises representative work on packet capture, traffic analysis and IPv6 header handling that is relevant to this project.

Table 2: Review of related work

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	S. McCanne and V. Jacobson, 1993 [1]	Kernel-resident BPF virtual machine for early packet filtering	Filtering cost versus earlier in-kernel filters; BSD kernels	Discards unwanted packets before copying; basis of libpcap filters	Filtering only; no decoding, statistics or presentation; predates IPv6
2	M. Roesch, 1999 [7]	Snort: libpcap-based rule-driven network intrusion detection	Detection by rules; alert output	Portable and lightweight; flexible rule language	Alert-oriented; not a per-layer teaching tool; no live dashboard

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
3	V. Paxson, 1999 [8]	Bro: event engine with a separate policy layer for real-time intrusion detection	Detection capability and real-time operation	Clean separation of mechanism and policy	Heavy to configure for simple inspection
4	L. Rizzo, 2012 [9]	netmap: shared-memory framework that bypasses much of the kernel network stack	Packets per second per core on 10 Gb/s hardware	Very high capture and forwarding rates	Needs special drivers; capture only; beyond lab scope
5	F. Gont et al., 2014 and 2021 [3], [4]	Standards analysis of IPv6 extension header chains and their operational impact	Qualitative operational and security analysis	Explains why Layer 4 information is hard to find and why chains must be complete in the first fragment	Documents the problem; provides no capture-tool implementation
6	chenjiandongx, n.d. [10]	Go terminal sniffer using gopacket/libpcap with per-process traffic views	CPU, memory and throughput compared with bandwhich and nethogs using iperf	Modern TUI; process attribution	Terminal only; no web streaming; not aimed at layer-by-layer teaching
7	I. (igor185), n.d. [11]	Course project sniffer on libpcap with Qt GUI and CLI	Functional demonstration	Extensible protocol classes; GUI and CLI	Desktop GUI; its documentation does not describe IPv6 chain traversal or a web stream
8	Wireshark and tcpdump projects [12], [13]	Full-featured dissectors and a minimal capture utility built on libpcap	Reference accuracy; widely used ground truth	Complete protocol coverage; trusted output	Not embeddable in a custom live pipeline; GUI is heavyweight

2.3 Research gap and justification of novelty

The surveyed work falls into three groups. Low-level facilities (BPF, netmap) provide fast capture but leave decoding and presentation to the programmer. Full systems (Snort, Bro, Wireshark) are powerful but are large, configuration-heavy or desktop-bound and do not expose a small, readable implementation of the layer-by-layer decoding that a networking course is meant to teach. Compact open-source sniffers are readable, but their documentation does not describe how they treat IPv6 extension header chains or the Linux cooked framing used for the “any” interface, and none of the surveyed projects discusses the self-capture feedback that appears when decoded packets are streamed to a service running on the captured host.

NetPulse does not propose a new protocol or filter. Its contribution is an engineered and evaluated combination that fills this gap: an extension-header-aware decoder, datalink-aware framing, a self-capture-proof streaming path with bounded blocking, and DNS query telemetry, all in a compact C++ codebase of roughly 1,600 lines with a browser dashboard. The novelty is demonstrated by comparing it with a baseline sniffer that lacks each of these features (Chapter 5).

3. PROPOSED SYSTEM

3.1 System architecture or topology

NetPulse is organised in three tiers, shown in Figure 1. The capture tier consists of three C++ programs. `sniffer` captures and prints decoded packets in the terminal with statistics, optional BPF filters and a payload hex dump. `sniffer_http` performs the same decoding but sends each packet as a JSON object to the backend. `dns_server` binds to UDP port 53, forwards queries to an upstream resolver and reports the parsed name, query type and client address to the backend. The service tier is a Node.js and Express backend on port 3000 that accepts the HTTP POST requests, exposes a REST interface and republishes the events over Socket.IO. The presentation tier is a browser dashboard that receives events over a WebSocket and shows a live packet list with TCP, UDP, DNS and ICMP filters, an inspection pane, summary counters including a DNS query counter, and a DNS monitor panel that lists the queries reported by `dns_server`. A Test DNS button runs one lookup through the forwarder end to end, so the whole path can be demonstrated on demand.

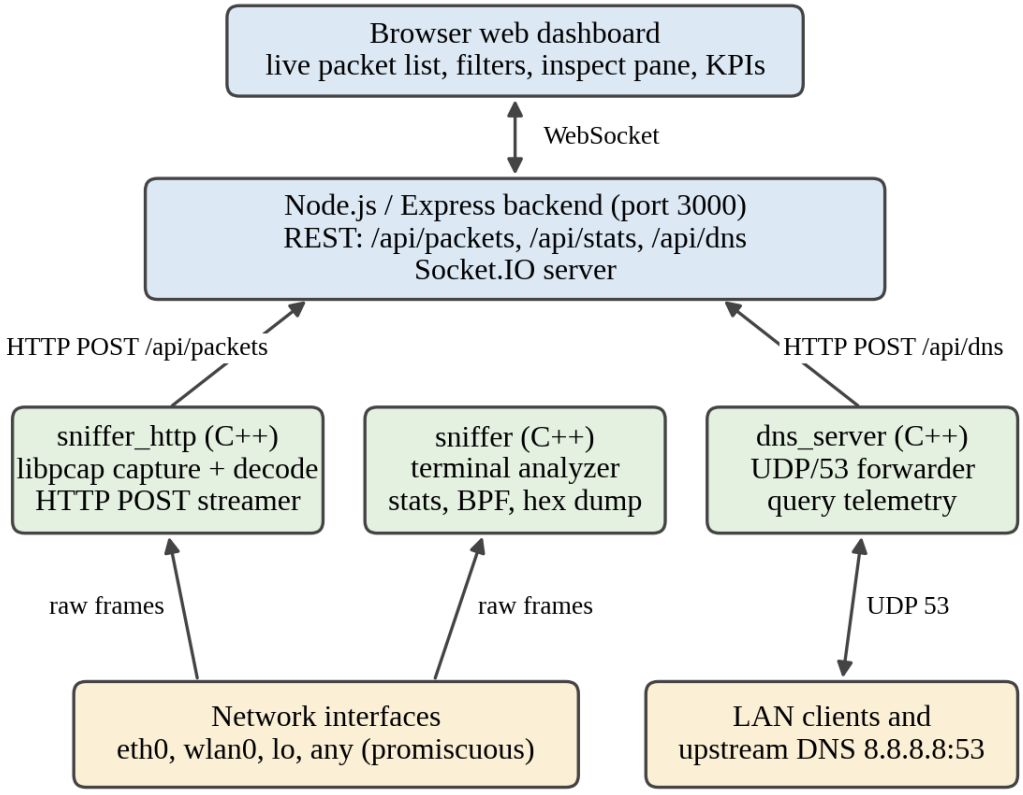


Figure 1: Architecture of the NetPulse system

The packet record posted by `sniffer_http` contains the source and destination addresses, the protocol name, the packet length, optional source and destination ports, the source and destination MAC addresses and a one-line description. The DNS record contains the queried domain, the client address and the query type. The backend turns each DNS report into a row in the packet table with protocol DNS; its destination is fixed to 8.8.8.8:53 in the backend and its length is estimated, so it is a summary record of the query and not a captured packet.

Decoupling the C++ capture programs from the web tier through HTTP and JSON keeps the capture code free of web-server dependencies and allows the dashboard to be replaced without touching it.

3.2 Baseline mechanism description

The baseline is a minimal libpcap sniffer of the kind commonly written as a first exercise, and it corresponds to the behaviour of the first version of this project before the defects described in Section 4.4 were fixed. It opens the interface in promiscuous mode, assumes a 14-byte Ethernet header, decodes IPv4 and IPv6, reads the IPv6 Next Header field of the base header as the transport protocol, installs no capture filter and posts every decoded packet to the backend. It therefore fails in three ways: on the “any” interface it reads cooked frames as Ethernet and decodes garbage, it reports every IPv6 packet that begins with an extension header as “Protocol 0” without ports or ICMPv6 type, and the HTTP POST requests it sends to port 3000 are captured and re-posted, so the volume of traffic grows without bound. For the measurements in Chapter 5 the baseline is built from the proposed source by disabling only the extension header traversal and the self-capture guard (Appendix A.5), so that these two features are isolated; the Ethernet-only failure is checked separately in Section 5.3.

3.3 Proposed mechanism: algorithm, pseudocode and flowchart

The proposed mechanism has three cooperating parts: datalink-aware framing, IPv6 header chain traversal and a self-capture guard. Algorithm 1 gives the per-packet pipeline and Figure 2 shows it as a flowchart.

Algorithm 1: Per-packet decode and dispatch

```
DECODE_AND_DISPATCH(packet, hdr, linktype)
 1  stats.total := stats.total + 1
 2  (ok, etype, l3, macs) := PARSE_LINK_LAYER(packet, hdr.caplen, linktype)
 3  if not ok then return
 4  if etype = IPv4 then
 5      check caplen >= l3 + 20 and IHL * 4 >= 20 and caplen >= l3 + IHL * 4
 6      proto := ip_p ;  l4 := l3 + IHL * 4
 7  else if etype = IPv6 then
 8      (proto, l4, ext) := WALK_IPV6_CHAIN(packet, hdr.caplen, l3)
 9  else if etype = ARP then
10      emit ARP request/reply with sender and target IP and MAC ;  return
11  else return
12  (sport, dport, detail) := DECODE_L4(proto, packet, l4, hdr.caplen)
13  if sport = P or dport = P then stats.self := stats.self + 1 ;  return
14  print one-line summary ;  POST_JSON(record) with 250 ms timeouts
```

Algorithm 2: IPv6 extension header traversal

```
WALK_IPV6_CHAIN(packet, caplen, l3)
 1  next := ip6_nxt ;  off := l3 + 40 ;  chain := empty list
 2  while off <= caplen do
 3      if next in {0, 43, 60, 135} then       // ext. headers
 4          if caplen < off + 2 then break
 5          hlen := (HdrExtLen + 1) * 8
 6      else if next = 44 then hlen := 8        // Fragment
 7      else if next = 51 then hlen := (PayloadLen + 2) * 4   // AH
 8      else break                              // upper-layer protocol reached
 9      if caplen < off + hlen then break
10      append name(next) to chain
11      next := packet[off] ;  off := off + hlen
12  return (next, off, chain)
```

Algorithm 3: Self-capture guard

```
SELF_CAPTURE_GUARD(P, userFilter)
 1  f := "not (tcp port " + P + ")"
 2  if userFilter is not empty then f := "(" + userFilter + ") and (" + f + ")"
 3  pcap_compile(f) ;  pcap_setfilter()          // dropped in the kernel
 4  in the callback: if sport = P or dport = P   // second line of defence
 5      then stats.self := stats.self + 1 ;  return
```

Algorithm 2 runs in time proportional to the number of extension headers in the packet, and every step is bounded by the captured length, so truncated or malformed packets end the traversal safely instead of reading past the buffer. When the traversal stops, the final Next Header value is the true upper-layer protocol, which allows TCP and UDP ports or the ICMPv6 type to be read at the resulting offset. The names of the headers that were skipped are shown next to the protocol so that the user can see that the packet carried, for example, a Hop-by-Hop option.

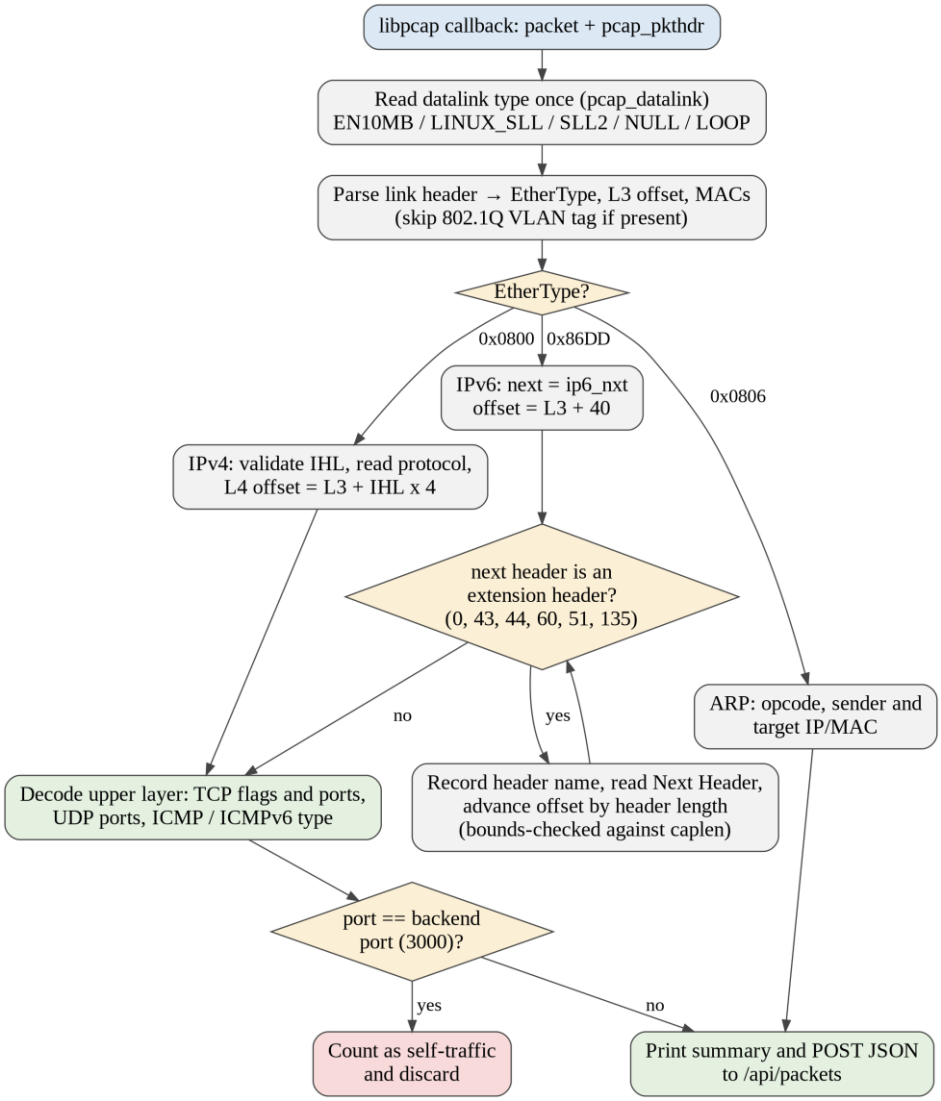


Figure 2: Flowchart of the packet decode and dispatch pipeline

3.4 Novelty statement

Table 3 states precisely what differs between the baseline and the proposed system.

Table 3: Baseline versus proposed mechanism

Aspect	Baseline	Proposed
IPv6 upper-layer discovery	Reads ip6_nxt of the base header; packets with extension headers appear as Protocol 0 with no ports	Traverses Hop-by-Hop, Routing, Fragment, Destination Options, AH and Mobility headers; reports the real protocol, ICMPv6 type and ports
Link layer	Ethernet only	Selected from the datalink type: Ethernet with 802.1Q, Linux cooked v1 and v2, loopback (NULL and LOOP)
Self-capture	None; POSTs to the backend are captured and re-posted	Kernel BPF expression excluding the backend port plus a user-space port check; excluded packets are counted
Backend coupling	Blocking sends with no timeout	250 ms send and receive timeouts; a single notice when the backend is unreachable while terminal capture continues
DNS visibility	None	UDP DNS forwarder that parses QNAME and QTYPE and reports each query
Observability	Packet lines only	Counters for captured, streamed, filtered and failed packets and kernel received and dropped counts at exit

3.5 Assumptions and design decisions

- **Platform.** Linux with libpcap and permission to capture; the executables receive `cap_net_raw` and `cap_net_admin` (and `cap_net_bind_service` for the DNS server) through the Makefile so that they can run without root.
- **Capture parameters.** Snapshot length 65535 bytes, promiscuous mode and a 1000 ms read timeout, so that complete frames are available to the decoder.
- **Decoupling through HTTP and JSON.** A plain HTTP POST over POSIX sockets was chosen over a library client to keep the C++ programs dependency-free and the protocol easy to inspect. The cost is a new TCP connection per packet, which is accepted for lab-scale traffic and analysed in Chapter 5.
- **Bounded blocking.** The capture callback must never wait indefinitely on the network, so the HTTP sockets use 250 ms timeouts (500 ms in the DNS server) and the upstream DNS wait is limited to 3 seconds.
- **Service names.** Well-known ports are mapped through a fixed table (19 ports in the terminal analyser) rather than by reading the system services database, so output is identical on every host.
- **Loopback families.** Address family values 2 (IPv4) and 10, 24 or 28 (IPv6 on Linux and the BSD family) are recognised in the loopback header.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 4: Software and hardware used

Component	Technology	Version / specification
Capture programs	C++17, libpcap, POSIX sockets, std::thread	g++ 13.3.0, libpcap 1.10.4 (TPACKET_V3)
Build and run	GNU Make, Bash scripts (start.sh, status.sh, stop.sh), systemd units	Flags: -Wall -Wextra -O2 -std=c++17 -pthread
Backend	Node.js, Express, Morgan, Socket.IO, CORS	Node.js v22.22.2; express 4.22.3, morgan 1.12.1, socket.io 4.8.3, cors 2.8.6
Dashboard	HTML, CSS and JavaScript served by Express	Modern browser
Measurement tools	Scapy (packet pool), tcpreplay (rate-controlled replay), tcpdump (ground-truth cross-check), Python 3 scripts	Scapy 2.8.0, tcpreplay 4.4.4, tcpdump 4.99.4
Operating system	Ubuntu 24.04.4 LTS for the evaluation; Kali Linux for development	Kernel 6.18.44
Hardware	Virtualised Linux environment	1 vCPU (Intel Xeon, 2.10 GHz), 4 GB RAM

4.2 Simulation or testbed setup

The evaluation runs on a single Linux host, as shown in Figure 3. A pool of 2,000 packets is built with Scapy [14] and injected onto the loopback interface at a controlled rate with tcpreplay [15]. The pool contains 40 % IPv6 packets with extension headers (nine header combinations built from Hop-by-Hop, Destination Options, Routing and Fragment headers, carrying TCP, UDP and ICMPv6), 20 % plain IPv6, 35 % IPv4 and 5 % ARP, and every packet has a unique source address so that each line of program output can be matched to the packet that caused it. The sniffer under test captures on the loopback interface and streams to a freshly started Node.js

backend on 127.0.0.1:3000, which puts the backend's own traffic on the captured interface: exactly the condition that causes self-capture.

Ground truth is known by construction from the pool metadata and was cross-checked independently with tcpdump [13]: its protocol-chain filters (`ip6 protochain`) matched 400 TCP, 489 UDP and 311 ICMPv6 IPv6 packets, identical to the counts in the metadata.

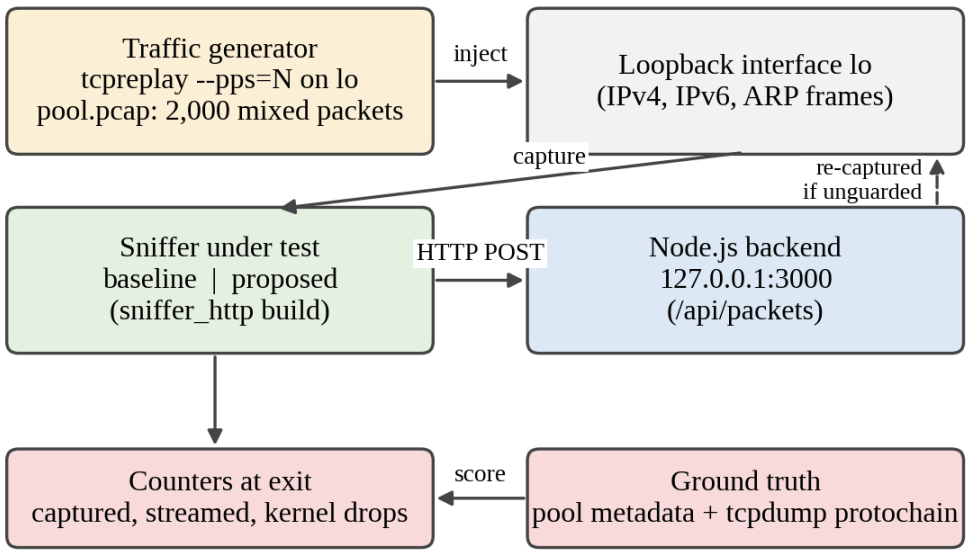


Figure 3: Evaluation testbed (single host, loopback)

Table 5: Configuration parameters

Parameter	Value	Where it is set
Snapshot length / promiscuous / read timeout	65535 B / on / 1000 ms	pcap_open_live in both sniffers
Default interface	eth0 (any, lo and wlan0 supported)	-i option
Backend host and port	127.0.0.1 : 3000	--host / --port, --backend-host / --backend-port
HTTP socket timeouts	250 ms (sniffer_http), 500 ms (dns_server)	SO_SNDTIMEO and SO_RCVTIMEO
Self-capture filter	not (tcp port 3000)	Built from the backend port
DNS listen port	UDP 53 (needs cap_net_bind_service)	-p option
Upstream resolver and wait	8.8.8.8 : 53, 3 s	-u, --upstream-port
Datagram buffer	4096 bytes	dns_server receive buffer
Hex dump length (verbose)	first 128 bytes	print_hex_dump in sniffer
Evaluation interface	lo	-i lo
Replay rate and volume	tcpreplay --pps=N --limit=N x 20 s	run_exp.py

4.3 Code structure and explanation of key modules

The repository is organised as follows. The C++ sources and build files are at the top level, the Node.js application is in `packet-streamer/`, and the systemd unit files are in `systemd/`.

```
Network-Traffic-Analyzer/
  sniffer.cpp         terminal packet analyser (625 lines)
  sniffer_http.cpp    HTTP streaming sniffer (576 lines)
  dns_server.cpp      UDP DNS forwarder with reporting (360 lines)
  Makefile            build, setcap and test targets (41 lines)
  start.sh status.sh stop.sh   service helper scripts
  systemd/            packet-streamer, packet-dns and packet-sniffer units
  packet-streamer/    Node.js backend, dashboard and test tools
```

Table 6: Key modules and their responsibilities

Module (file)	Responsibility
parse_link_layer (both sniffers)	Reads the datalink type once and returns EtherType, layer-3 offset and MAC strings for Ethernet (skipping an 802.1Q tag), Linux cooked v1 and v2, and loopback.
packet_callback (both sniffers)	libpcap callback with three branches: IPv4 (validates IHL, decodes TCP, UDP, ICMP), IPv6 (runs the chain traversal, then decodes TCP, UDP, ICMPv6) and ARP (opcode, sender and target IP and MAC).
IPv6 chain traversal	Loop over extension headers using the length rules of each header type; bounded by the captured length; records the headers seen.
Name tables	protocol_name, port_service, icmp_type_name and icmpv6_type_name translate numbers into readable names (19 ports, 8 ICMP types and 15 ICMPv6 types in the terminal analyser).
print_hex_dump (sniffer)	Verbose mode: offset, hexadecimal and ASCII view of up to 128 payload bytes.
Statistics (sniffer)	Totals per IP version and protocol and the kernel received and dropped counts from pcap_stats, printed on exit.
post_packet_to_backend (sniffer_http)	Opens a TCP socket with timeouts, builds the JSON body with escape_json and sends an HTTP/1.1 POST to /api/packets; maintains streamed and error counters.
Self-capture guard (sniffer_http)	Composes the BPF expression and applies the second check on ports inside the callback.
parse_dns_query (dns_server)	Validates the 12-byte header and QDCOUNT, reads length-prefixed labels (handling compression pointers and malformed lengths) and the two-byte QTYPE.
process_query (dns_server)	Runs in a detached thread per query: parse, report to /api/dns, forward to the upstream resolver, wait up to 3 s and relay the answer to the client.
Makefile and scripts	Builds the three binaries, applies Linux capabilities and provides a test target. start.sh builds missing binaries and starts three systemd services (backend, DNS server, sniffer); status.sh and stop.sh inspect and stop them. Paths are hard-coded to /home/kali.

Module (file)	Responsibility
packet-streamer/ (Node.js)	Express server with Morgan logging and Socket.IO. REST endpoints: /api/packets and /api/stats, /api/dns (reports from dns_server) and /api/dns/test (runs a lookup through the forwarder). Ring buffers hold the last 300 packets and 100 DNS queries. The single-page dashboard has a live list, TCP/UDP/DNS/ICMP filters, an inspection pane, KPI counters, a DNS monitor panel and Test DNS and Sim Packet buttons. Sim Packet posts synthetic records (about one click in four runs the real DNS test instead) and was not used in the measurements. test_socket.js and simulate_packets.py are test tools.

4.4 Challenges faced and how they were solved

Table 7: Challenges and solutions

#	Challenge	Cause	Solution
1	Many IPv6 packets shown as “Protocol 0”	The base header’s Next Header field is 0 (Hop-by-Hop) for MLD, Router Alert and Neighbor Discovery traffic	Extension header traversal that reaches the real protocol, ICMPv6 type and ports
2	Garbage output on the “any” interface	libpcap delivers Linux cooked frames (SLL) rather than Ethernet	Datalink-aware parser for EN10MB, LINUX_SLL, LINUX_SLL2, NULL and LOOP
3	Traffic storm while streaming	HTTP POSTs to port 3000 are captured and re-posted	Kernel BPF exclusion of the backend port plus a user-space check, with a counter
4	Needing root to capture	Raw capture needs CAP_NET_RAW and CAP_NET_ADMIN	Makefile applies file capabilities; passwordless sudo was enabled only on the disposable lab VM
5	Capture stalls if the backend is down or slow	Blocking connect and send inside the callback	250 ms timeouts, a one-time notice, and continued terminal output
6	Binding the DNS server to port 53	Ports below 1024 are privileged	cap_net_bind_service on the binary and a clear hint on EACCES
7	Malformed or truncated packets	Short captures, bad IHL, bad label lengths	Length checks before every header access in the decoders and the DNS parser
8	Valid JSON from arbitrary strings	Packet descriptions and DNS labels may contain special characters	escape_json handling quotes, backslashes and control characters

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

All measurements were taken on the loopback interface of a single-vCPU virtual host, so absolute throughput figures are specific to that environment and the comparison between the baseline and the proposed build is the finding. The sniffer, the backend and the replay tool share the one CPU, which makes the high-load scenario deliberately demanding. The experiment scripts and the raw per-run data accompany this report so that the runs can be repeated on other hosts, and every counter used below is printed by the programs themselves when they exit.

The backend used in the measurements is the version from the first commit of the repository. Later commits add DNS monitoring to the backend and dashboard but do not change how packets are accepted, and the C++ programs and the Makefile are unchanged, so the measurements exercise the same capture code as the current repository.

5.1 Performance metrics and why they were chosen

Table 8: Metrics

ID	Metric	Definition	Source
M1	IPv6 identification accuracy (%)	Share of IPv6 packets carrying at least one extension header whose reported protocol, and ports where applicable, match the ground truth	Program output versus pool metadata, cross-checked with tcpdump
M2	Backend amplification factor	Packets streamed to the backend divided by the packets offered by the generator; the ideal value is 1	streamed_to_backend counter and tcpreplay packet count
M3	Kernel drop rate (%)	Dropped divided by received packets reported by libpcap (on loopback each frame is counted twice, which does not change the ratio)	pcap_stats printed by the measurement build
M4	Delivery ratio (%)	Streamed divided by captured minus filtered self-traffic	Counters printed by sniffer_http
M5	Traffic coverage (%)	IPv6 pool packets captured and decoded divided by IPv6 pool packets offered (60 % of the offered packets)	Program output and tcpreplay packet count

M1 measures the correctness defect that motivated the project. M2 captures the feedback storm, because a feedback loop multiplies the number of packets the backend receives. M3 shows whether the capture path keeps up with the offered load, which is what the per-packet connection design puts at risk. M4 shows how much of what reached the streaming code was actually written to the backend, and M5 shows what share of the offered traffic the program analysed end to end, which makes the two builds comparable even when both are saturated.

5.2 Scenarios and parameter values

Table 9: Scenarios

Scenario	Offered load	Traffic	Duration, repeats
S1 (low load)	50 packets/s (measured 50.0)	Replay of the 2,000-packet pool: 40 % IPv6 with extension headers, 20 % plain IPv6, 35 % IPv4, 5 % ARP	20 s, 5 runs
S2 (medium load)	1,000 packets/s (measured 998.5)	Replay of the 2,000-packet pool: 40 % IPv6 with extension headers, 20 % plain IPv6, 35 % IPv4, 5 % ARP	20 s, 5 runs
S3 (high load)	10,000 packets/s (measured 9,983)	Replay of the 2,000-packet pool: 40 % IPv6 with extension headers, 20 % plain IPv6, 35 % IPv4, 5 % ARP	20 s, 5 runs

In every scenario the two builds run one after the other on the same interface with a freshly started backend, in alternating order within each repeat. The offered packet count is taken from the tcpreplay summary, and each run lasts 20 s of replay followed by a 2 s drain before the sniffer is stopped. The baseline is derived from the proposed source by the two changes described in Appendix A.5, and both builds print the same counters.

5.3 Results: tables and graphs, baseline vs proposed

Table 10: Results (mean ± standard deviation over five runs)

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
S1 (50 pkt/s)	M1 IPv6 identification accuracy (%)	0.0 ± 0.0	100.0 ± 0.0	+100.0 pp
S1 (50 pkt/s)	M2 Backend amplification factor	36.80 ± 0.70	1.00 ± 0.00	97.3 % lower
S1 (50 pkt/s)	M3 Kernel drop rate (%)	87.25 ± 0.04	0.00 ± 0.00	100.0 % lower
S1 (50 pkt/s)	M5 Traffic coverage (%)	13.7 ± 0.7	100.0 ± 0.0	+86.3 pp
S2 (1,000 pkt/s)	M1 IPv6 identification accuracy (%)	0.0 ± 0.0	100.0 ± 0.0	+100.0 pp
S2 (1,000 pkt/s)	M2 Backend amplification factor	1.95 ± 0.02	1.00 ± 0.00	48.8 % lower
S2 (1,000 pkt/s)	M3 Kernel drop rate (%)	87.98 ± 0.09	0.00 ± 0.00	100.0 % lower
S2 (1,000 pkt/s)	M5 Traffic coverage (%)	8.1 ± 0.1	100.0 ± 0.0	+91.9 pp
S3 (10,000 pkt/s)	M1 IPv6 identification accuracy (%)	0.0 ± 0.0	100.0 ± 0.0	+100.0 pp
S3 (10,000 pkt/s)	M2 Backend amplification factor	0.20 ± 0.00	0.20 ± 0.00	none (both at ceiling)
S3 (10,000 pkt/s)	M3 Kernel drop rate (%)	91.98 ± 0.05	78.98 ± 0.32	14.1 % lower
S3 (10,000 pkt/s)	M5 Traffic coverage (%)	3.1 ± 0.2	19.8 ± 0.5	+16.8 pp

Improvement is the gain in percentage points (pp) for M1 and M5 and the relative reduction for M2 and M3. SD is the sample standard deviation of the five runs. M4 (delivery ratio) was 99.8–100 % in every cell: 99.80 % for the baseline in all three scenarios and 100.00 %, 99.87 % and 99.80 % for the proposed build in S1, S2 and S3.

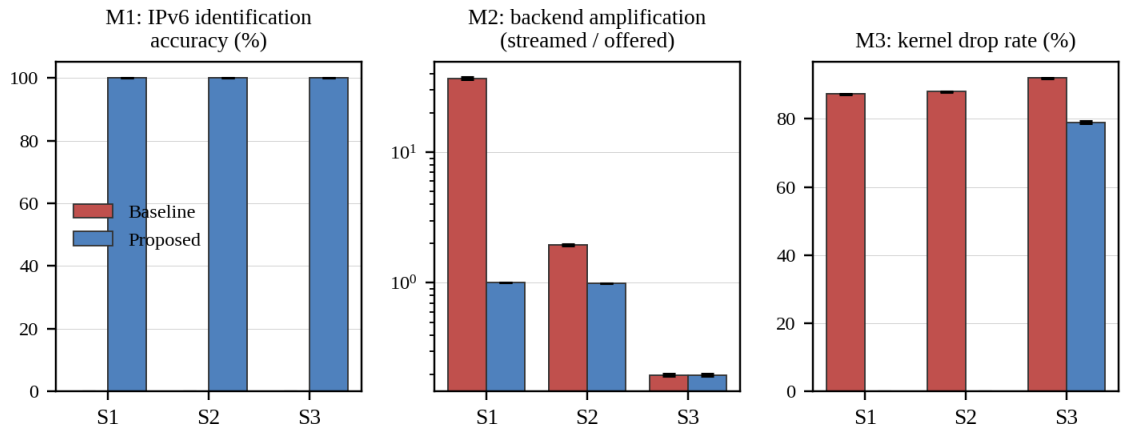


Figure 4: Baseline versus proposed for M1, M2 and M3 (mean ± SD, five runs; M2 on a logarithmic axis)

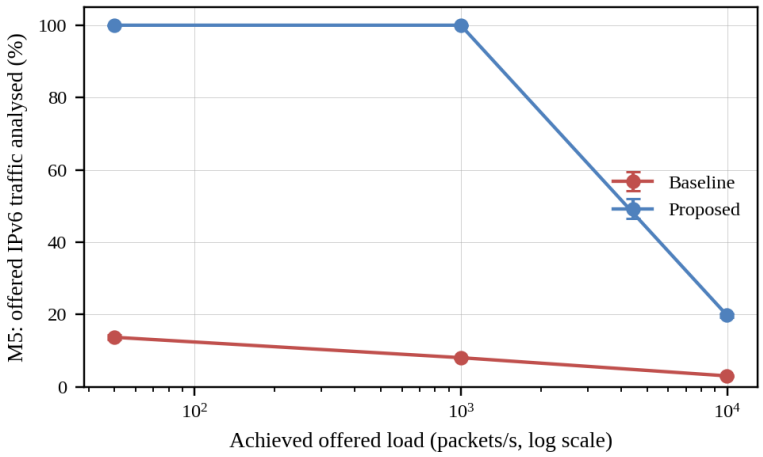


Figure 5: Share of offered IPv6 traffic analysed (M5) against achieved offered load

Two supplementary checks support claims made in Chapters 3 and 5. They were run with the HTTP streaming disabled so that only the decoder is exercised.

Table 11: Supplementary checks

Check	Decoder	Result
Capture on the any interface (Linux cooked frames); 1,000 packets per run, 5 runs, 600 IPv6 packets per run	Ethernet-only parser	0 of 600 decoded correctly in every run
Same	Datalink-aware (proposed)	600 of 600 decoded correctly in every run
200 non-first IPv6 fragments on lo	Proposed	200 of 200 reported as UDP with ports taken from fragment data (false ports)

5.4 Analysis and discussion

Identification accuracy (M1). Every extension-header packet that reached the baseline was misreported, in every run and at every load (0.0 ± 0.0 %), whereas the proposed build identified

all of them (100.0 ± 0.0 %). For three sample packets the baseline printed HOPOPT, HOPOPT and Proto 60, where the proposed build printed ICMPv6 (MLDv2 Report), TCP (HTTP) with ports and UDP (DNS) with ports (Appendix B). Plain IPv6 packets in the same pool were identified correctly by both builds in every run, so the failure is specific to extension headers. Because every extension-header packet in the pool starts with an extension header, a zero for the baseline is what the design predicts; on traffic with fewer such packets its overall IPv6 accuracy would be higher, but it would still miss all Hop-by-Hop traffic such as MLD reports.

Amplification (M2). Without the guard, every packet posted to the backend is itself captured and posted again. At 50 packets/s the baseline streamed about 36,800 packets for 1,000 offered, an amplification of 36.80 ± 0.70, whereas the proposed build streamed exactly the packets offered (1.00 ± 0.00). The storm continues until the capture thread reaches its own processing ceiling: the baseline captured between 36,900 and 39,400 packets per run however little traffic was offered, which is why its amplification falls to 1.95 at 1,000 packets/s and to 0.20 at 10,000 packets/s. At 10,000 packets/s both builds are at the ceiling, so M2 is equal, and the difference appears in what the ceiling is spent on.

Kernel drops and coverage (M3, M5). The storm also fills the kernel buffer. The baseline lost 87–92 % of frames in every scenario and analysed only 13.7 %, 8.1 % and 3.1 % of the offered IPv6 traffic. The proposed build lost no frames at 50 and 1,000 packets/s and analysed all of the traffic. At 10,000 packets/s it lost 78.98 ± 0.32 % of frames and analysed 19.8 ± 0.5 % of the offered traffic, about 6.5 times as much as the baseline, because its capacity goes to the offered traffic instead of to its own.

Delivery (M4) and the throughput ceiling. Of the packets that reached the streaming code, 99.8–100 % were written to the backend in every run; the remaining 0.2 % (about 75 per run at saturation) are failed or timed-out sends when the single CPU is saturated. Loss therefore occurs in the kernel buffer and not in the HTTP path. The ceiling of roughly 1,800 packets/s (about 39,600 packets in a 22-second window) is set by the per-packet TCP connection, the 250 ms worst-case waits, and the fact that capture, backend and replay compete for one CPU. A persistent connection or batching, proposed in Section 6.3, is the obvious remedy, and the absolute ceiling should be re-measured on other hardware.

Link layer. On the any interface, where libpcap delivers Linux cooked frames, an Ethernet-only parser decoded 0 of 600 IPv6 packets in each of five runs, whereas the datalink-aware decoder decoded all 600 in each run (Table 11).

5.5 Critical evaluation: strengths, weaknesses and failure conditions

Strengths. The decoder identified every extension-header packet in the measured traffic, the exit counters made each run verifiable, run-to-run variation was small (standard deviations below 1 percentage point for M1, M3 and M5 in every scenario), blocking time is bounded, and the code base is small enough to be read and defended in a viva.

Weaknesses and conditions under which the scheme fails. Table 12 lists the conditions identified by reviewing the source code. Two of them were also confirmed by measurement: the false ports on non-first fragments (Table 11) and the throughput ceiling (Section 5.4).

Table 12: Failure conditions and possible mitigations

Condition	Effect	Mitigation
IPv6 fragments other than the first	The traversal does not check the fragment offset, so bytes after the Fragment header are read as a UDP or TCP header. Measured: 200 of 200 such fragments were reported with false ports	Skip L4 decoding when the fragment offset is non-zero
Header chain longer than the captured data	Traversal stops early; the last header seen is reported as the protocol	Larger snapshot length; flag the packet as truncated
Encrypted payloads (ESP)	Reported as ESP; ports are unavailable	None possible without keys
Other traffic on TCP port 3000	The kernel filter and the user-space check discard it together with the system's own traffic	Use a dedicated, unusual port or match the backend socket pair; the user-space check also discards UDP port 3000
High packet rates	A new TCP connection per packet limits throughput. Measured: ceiling of roughly 1,800 packets/s on one vCPU and 79 % kernel drops at 10,000 packets/s	Persistent connection or batching and a queue between capture and sender thread
Delivery accounting	A packet is counted as streamed when send() returns a positive value; the HTTP response is never read	Read the status line or use acknowledgements
No capture timestamp in the record	The dashboard cannot show the true capture time or measure latency	Add the pcap timestamp to the JSON
Special bytes in strings	escape_json in the DNS server escapes quotes, backslash, newline, carriage return and tab only; other control or non-ASCII bytes in a crafted label could give invalid JSON	Escape all control characters and validate UTF-8
DNS server exposure	Binds to all addresses and forwards for any client, so it behaves as an open resolver; the packet-dns systemd unit now starts it permanently on port 53; one detached thread per query is unbounded	Bind to the lab interface, add rate limits and a thread pool
DNS rows built by the backend	The row for a reported query has a fixed destination of 8.8.8.8:53 and an estimated length, so it can be mistaken for a captured packet	Carry the real upstream and length in the report, or label such rows as reports
Unauthenticated dashboard	Anyone who can reach the backend can read or inject events	Authentication, TLS and a loopback-only bind

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

NetPulse delivers a complete capture, decode and live-streaming pipeline in about 1,560 lines of C++ across three programs, plus a Node.js backend and browser dashboard. The decoders handle five datalink types, IPv4, IPv6 with six extension header types, ARP, TCP with six flag bits, UDP, eight ICMP types and fifteen ICMPv6 types, and the DNS forwarder extracts the query name and type from live traffic. The three defects of the baseline sniffer were removed: the "Protocol 0" misclassification, the failure on cooked captures and the self-capture feedback storm.

In the evaluation (five 20-second runs per scenario on loopback), the baseline identified 0 % of the extension-header packets against 100 % for the proposed build in every scenario. Backend amplification fell from 36.8 to 1.0 at 50 packets/s and from 1.95 to 1.00 at 1,000 packets/s, and kernel drops fell from 87–88 % to 0 % at those loads. At 10,000 packets/s both builds reached the same processing ceiling of roughly 1,800 packets/s on the single-vCPU test host, and the proposed build analysed 19.8 % of the offered IPv6 traffic against 3.1 % for the baseline. On the "any" interface the datalink-aware decoder decoded all 600 IPv6 packets in each run, while an Ethernet-only parser decoded none.

6.2 Limitations

The system does not reassemble streams, defragment packets or track flows. Fragment offsets are not examined, so non-first IPv6 fragments can be misdecoded. The streaming path opens one TCP connection per packet, delivery is not confirmed, packets carry no capture timestamp, and the filter that prevents feedback also hides other traffic on the backend port. The DNS forwarder is UDP-only and unthrottled, and the dashboard has no authentication or encryption. These limits are quantified or explained in Section 5.5.

6.3 Future enhancements

- Add flow tracking with 5-tuples, per-host and per-protocol bandwidth, top talkers and TCP handshake, retransmission and round-trip-time analysis.
- Replace per-packet connections with a persistent connection or batching, and decouple capture from sending with a ring buffer and a sender thread.
- Add an offline mode that reads a pcap file, so that evaluations can be repeated exactly, and export captures in pcap format.
- Check the IPv6 fragment offset, handle IPv4 fragmentation, and parse DNS responses to report resolution latency and response codes.
- Add anomaly detection for SYN scans and ARP spoofing, with alerts on the dashboard.
- Add authentication and TLS to the dashboard, bind services to the lab interface and carry the capture timestamp in every record.

7. REFERENCES

References (IEEE style)

[1] S. McCanne and V. Jacobson, “The BSD packet filter: A new architecture for user-level packet capture,” in *Proc. USENIX Winter Technical Conf.*, San Diego, CA, USA, Jan. 1993, pp. 259–269.

[2] S. Deering and R. Hinden, “Internet Protocol, Version 6 (IPv6) Specification,” IETF, RFC 8200, Jul. 2017.

[3] F. Gont, N. Hilliard, G. Doering, W. Kumari, G. Huston, and W. Liu, “Operational Implications of IPv6 Packets with Extension Headers,” IETF, RFC 9098, Sep. 2021.

[4] F. Gont, V. Manral, and R. Bonica, “Implications of Oversized IPv6 Header Chains,” IETF, RFC 7112, Jan. 2014.

[5] A. Conta, S. Deering, and M. Gupta, Ed., “Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification,” IETF, RFC 4443, Mar. 2006.

[6] P. Mockapetris, “Domain Names - Implementation and Specification,” IETF, RFC 1035, Nov. 1987.

[7] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. 13th USENIX Systems Administration Conf. (LISA)*, Seattle, WA, USA, Nov. 1999, pp. 229–238.

[8] V. Paxson, “Bro: A system for detecting network intruders in real-time,” *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, Dec. 1999.

[9] L. Rizzo, “netmap: A novel framework for fast packet I/O,” in *Proc. USENIX Annu. Technical Conf. (ATC)*, Boston, MA, USA, Jun. 2012, pp. 101–112.

[10] chenjiandongx, “sniffer: A modern alternative network traffic sniffer,” GitHub repository. [Online]. Available: https://github.com/chenjiandongx/sniffer (accessed Oct. 26, 2026).

[11] igor185, “sniffer: Final project in OS course, traffic capture with libpcap and Qt,” GitHub repository. [Online]. Available: https://github.com/igor185/sniffer (accessed Oct. 26, 2026).

[12] Wireshark Foundation, “Wireshark network protocol analyzer.” [Online]. Available: https://www.wireshark.org (accessed Oct. 26, 2026).

[13] The Tcpdump Group, “TCPDUMP and LIBPCAP.” [Online]. Available: https://www.tcpdump.org (accessed Oct. 26, 2026).

[14] P. Biondi and the Scapy community, “Scapy: Packet crafting for Python.” [Online]. Available: https://scapy.net (accessed Oct. 26, 2026).

[15] A. Turner et al., “Tcpreplay: Pcap editing and replaying utilities.” [Online]. Available: https://tcpreplay.appneta.com (accessed Oct. 26, 2026).

[16] A. Pratap, “Network-Traffic-Analyzer (NetPulse),” GitHub repository. [Online]. Available: https://github.com/GitWithAdi/Network-Traffic-Analyzer (accessed Oct. 26, 2026).

Appendix A: Important source code or configuration listings

The listings are abridged excerpts from the repository; long lines are wrapped for the page.

A.1 IPv6 extension header traversal (sniffer_http.cpp)

```
uint8_t next_proto = ip6->ip6_nxt;
size_t cur_offset = l3_offset + sizeof(struct ip6_hdr);
bool has_hopopt = false;
while (cur_offset <= header->caplen) {
    if (next_proto == IPPROTO_HOPOPTS) {
        has_hopopt = true;
        if (header->caplen < cur_offset + 2) break;
        uint8_t nxt = packet[cur_offset];
        uint8_t len = packet[cur_offset + 1];
        size_t hlen = (static_cast<size_t>(len) + 1) * 8;
        if (header->caplen < cur_offset + hlen) break;
        next_proto = nxt;
        cur_offset += hlen;
    } else if (next_proto == IPPROTO_ROUTING ||
               next_proto == IPPROTO_DSTOPTS || next_proto == 135) {
        /* same pattern: hlen = (len + 1) * 8 */
    } else if (next_proto == IPPROTO_FRAGMENT) {
        if (header->caplen < cur_offset + 8) break;
        uint8_t nxt = packet[cur_offset];
        next_proto = nxt;
        cur_offset += 8;
    } else if (next_proto == IPPROTO_AH) {
        /* same pattern: hlen = (len + 2) * 4 */
    } else {
        break;
    }
}
```

A.2 Self-capture guard (sniffer_http.cpp)

```
std::string final_filter =
    "not (tcp port " + std::to_string(g_backend_port) + ")";
if (!custom_filter.empty()) {
    final_filter = "(" + custom_filter + ") and (" + final_filter + ")";
}
struct bpf_program fp;
if (pcap_compile(g_pcap_handle, &fp, final_filter.c_str(), 1,
                 PCAP_NETMASK_UNKNOWN) != -1) {
    pcap_setfilter(g_pcap_handle, &fp);
    pcap_freecode(&fp);
}

/* inside packet_callback, after decoding the ports */
if (src_port == g_backend_port || dst_port == g_backend_port) {
    g_stats.filtered_self_traffic++;
    return;
}
```

A.3 DNS question parsing (dns_server.cpp)

```
size_t offset = 12;                    // DNS header is 12 bytes
std::string domain = "";
while (offset < length) {
    uint8_t label_len = buffer[offset++];
    if (label_len == 0) break;         // end of QNAME
    if ((label_len & 0xC0) == 0xC0) {  // compression pointer
        if (offset >= length) return result;
        offset++;
        break;
    }
    if (offset + label_len > length) return result;   // malformed
    if (!domain.empty()) domain += ".";
    for (size_t i = 0; i < label_len; ++i)
        domain += static_cast<char>(buffer[offset + i]);
    offset += label_len;
}
if (offset + 2 <= length)              // QTYPE follows QNAME
    result.qtype = (static_cast<uint16_t>(buffer[offset]) << 8) |
                    buffer[offset + 1];
```

A.4 Build configuration (Makefile)

```
CXX ?= g++
CXXFLAGS ?= -Wall -Wextra -O2 -std=c++17 -pthread
LDFLAGS ?= -lpcap
all: $(TARGET) $(HTTP_TARGET) $(DNS_TARGET) setcap
setcap: $(TARGET) $(HTTP_TARGET) $(DNS_TARGET)
      @sudo setcap cap_net_raw,cap_net_admin=eip $(TARGET) 2>/dev/null || true
      @sudo setcap cap_net_raw,cap_net_admin=eip $(HTTP_TARGET) 2>/dev/null || true
      @sudo setcap cap_net_bind_service=+ep $(DNS_TARGET) 2>/dev/null || true
```

A.5 Deriving the baseline for the evaluation

The baseline binary used in Chapter 5 is built from `sniffer_http.cpp` by a script (`build_variants.py`) with two compile-time switches, `-DBASELINE_NO_EH=1` and `-DBASELINE_NO_GUARD=1`. The first disables the extension header loop in A.1, so the Next Header field of the base header is used directly as the protocol. The second removes the filter construction and the user-space port check in A.2, so nothing prevents self-capture. Everything else is identical. Both builds also include one small addition so that M3 can be measured: after the capture loop ends, `pcap_stats` is called and “Kernel Received” and “Kernel Dropped” are printed. This does not change the behaviour of the programs.

A.6 Experiment scripts

Table A1: Scripts supplied with the report

File	Role
gen_pool.py	Builds pool.pcap (2,000 packets) and meta.json, the ground truth for every source address
build_variants.py	Derives the baseline and proposed measurement builds from sniffer_http.cpp
run_exp.py	Runs the 30 trials (3 scenarios, 2 builds, 5 repeats), scores the output and writes results.jsonl
run_aux.py	Link-layer check on the any interface and the non-first fragment check
analyze.py	Computes mean and standard deviation and draws Figures 4 and 5
results_raw.csv	Raw counters of every run

```
python3 gen_pool.py 2000
python3 build_variants.py ../sniffer_http.cpp
python3 run_exp.py results.jsonl 20 5 S1,S2,S3
python3 run_aux.py
python3 analyze.py results.jsonl
```

Appendix B: Additional screenshots, packet captures, raw result tables

B.1 Sample output for the same three packets

The packets are (1) a Hop-by-Hop header carrying an MLDv2 report, (2) a Hop-by-Hop header carrying a TCP SYN to port 80 and (3) a Destination Options header carrying a UDP datagram to port 53.

```
Baseline
[#1] IPv6 (HOPOPT) fd00::5 -> ::1 [70B]
[#2] IPv6 (HOPOPT) fd00::6 -> ::1 [82B]
[#3] IPv6 (Proto 60) fd00::7 -> ::1 [78B]

Proposed
[#1] IPv6 (ICMPv6 (MLDv2 Report)) fd00::5 -> ::1 [70B]
[#2] IPv6 (TCP (HTTP)) fd00::6:40001 -> ::1:80 [82B]
[#3] IPv6 (UDP (DNS)) fd00::7:40002 -> ::1:53 [78B]
```

B.2 Raw per-run values

Table B1: Raw per-run values behind Table 10

Scenario	Metric	Build	Run 1	Run 2	Run 3	Run 4	Run 5
S1	M1	Baseline	0.0	0.0	0.0	0.0	0.0
S1	M1	Proposed	100.0	100.0	100.0	100.0	100.0
S1	M2	Baseline	37.12	36.78	37.47	37.00	35.64
S1	M2	Proposed	1.00	1.00	1.00	1.00	1.00
S1	M3	Baseline	87.25	87.21	87.23	87.26	87.31
S1	M3	Proposed	0.00	0.00	0.00	0.00	0.00

Scenario	Metric	Build	Run 1	Run 2	Run 3	Run 4	Run 5
S1	M5	Baseline	14.0	14.3	13.2	12.8	14.3
S1	M5	Proposed	100.0	100.0	100.0	100.0	100.0
S2	M1	Baseline	0.0	0.0	0.0	0.0	0.0
S2	M1	Proposed	100.0	100.0	100.0	100.0	100.0
S2	M2	Baseline	1.98	1.96	1.92	1.96	1.93
S2	M2	Proposed	1.00	1.00	1.00	1.00	1.00
S2	M3	Baseline	87.94	87.99	87.93	88.14	87.91
S2	M3	Proposed	0.00	0.00	0.00	0.00	0.00
S2	M5	Baseline	8.2	8.1	7.8	8.2	8.1
S2	M5	Proposed	100.0	100.0	100.0	100.0	100.0
S3	M1	Baseline	0.0	0.0	0.0	0.0	0.0
S3	M1	Proposed	100.0	100.0	100.0	100.0	100.0
S3	M2	Baseline	0.20	0.20	0.19	0.19	0.20
S3	M2	Proposed	0.20	0.20	0.20	0.20	0.19
S3	M3	Baseline	91.91	91.97	92.01	92.03	91.99
S3	M3	Proposed	78.82	78.66	78.95	78.98	79.51
S3	M5	Baseline	3.1	3.4	2.8	3.0	3.1
S3	M5	Proposed	20.0	20.3	19.9	19.8	19.0

B.3 Dashboard screenshots

The screenshots were taken on the Kali virtual machine (address 10.0.2.15) while it was browsing, so they show real captured traffic.

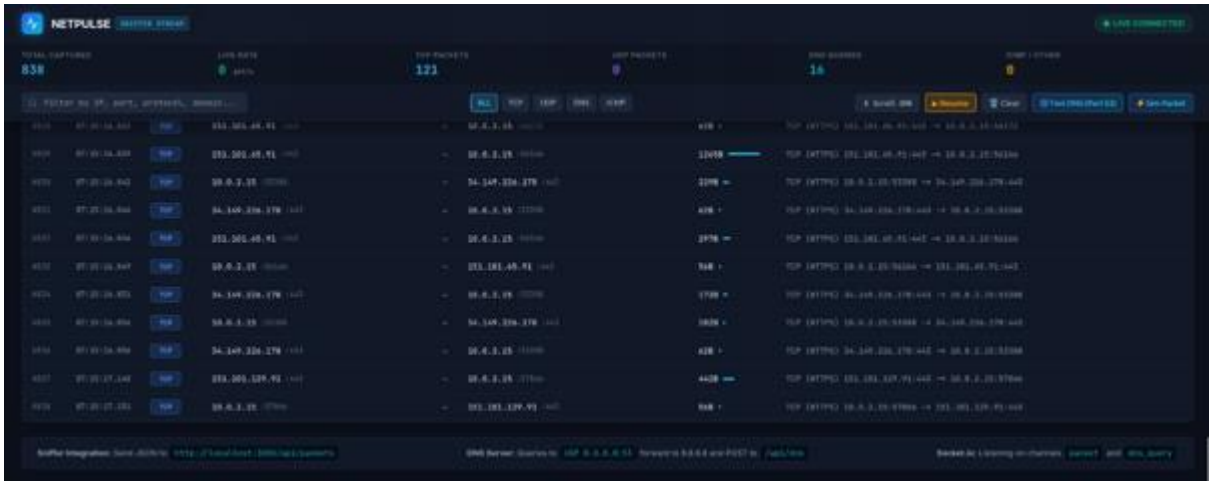


Figure B1: Live dashboard showing captured HTTPS (TCP) and DNS (UDP) packets

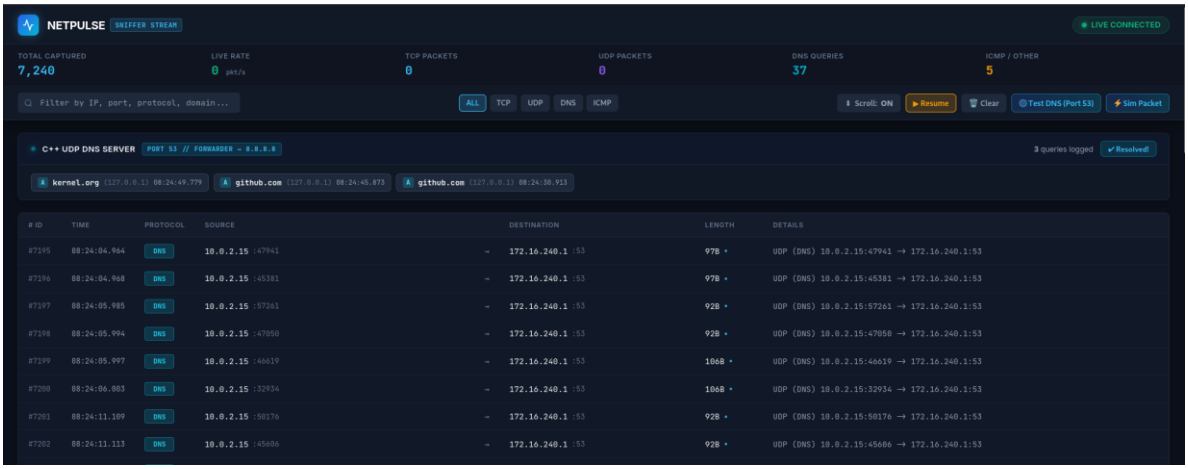


Figure B2: Dashboard with the DNS monitor panel, DNS query counter and DNS filter

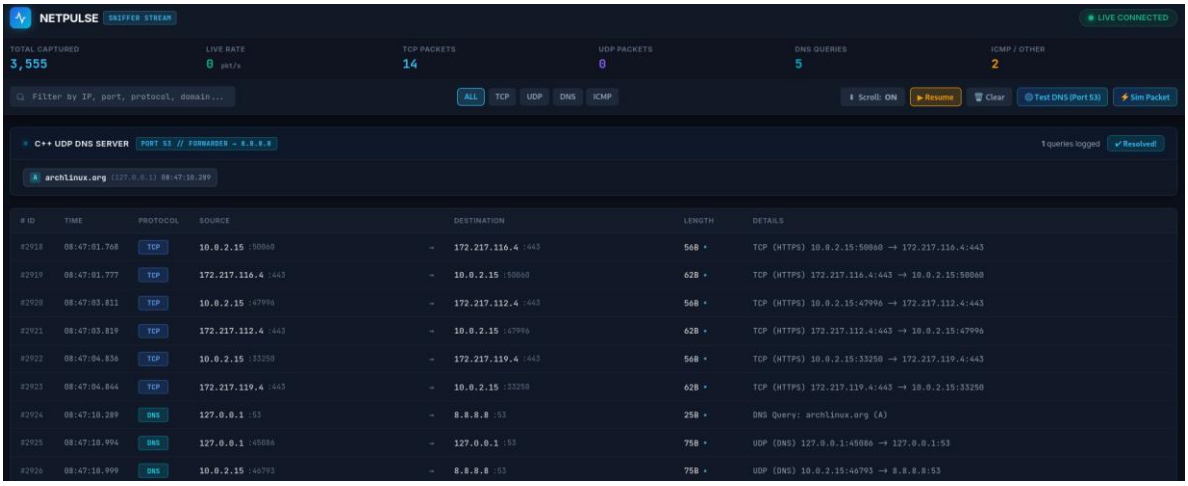


Figure B3: After pressing Test DNS: the query row for archlinux.org and the packets of the lookup